5

Unit 5: State Machines, Memory & PLDs

Digital Logic Design

CHANDRAS EDU

Never stop learning

Outline

5.1 Moore vs. Mealy Finite State Machines
5.2 State Diagrams & State Tables
5.3 State Reduction & State Assignment
5.4 FSM Design Procedure: Sequence Detector
5.5 Programmable Logic Devices
5.6 Memory: RAM and ROM
5.7 FSM Case Studies: Traffic Light & Vending Machine

5.1 Moore vs. Mealy Finite State Machines

IN SIMPLE TERMS

A Moore machine’s output depends only on which state it is in; a Mealy machine’s output also depends on the input arriving now.

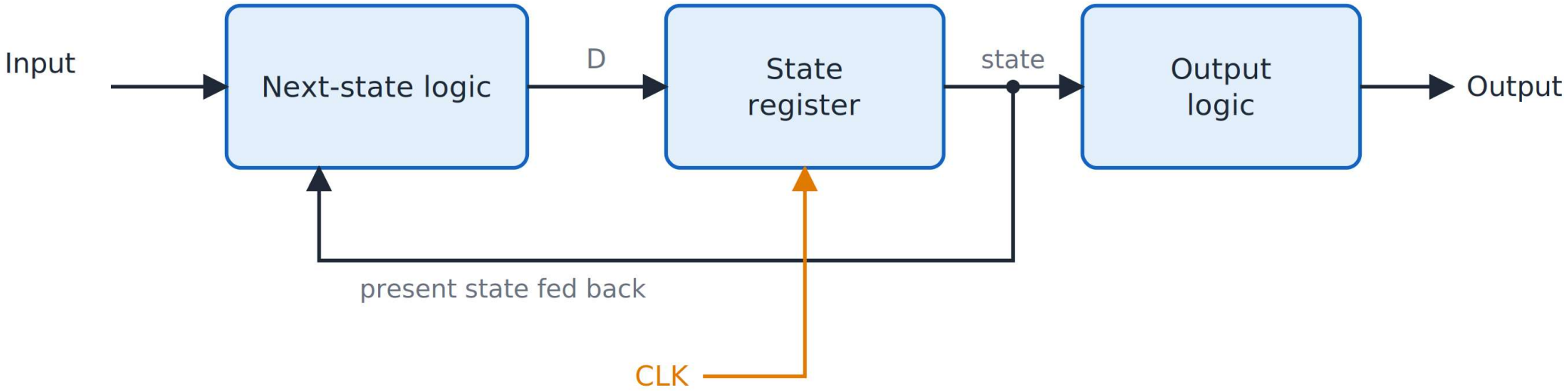
From sequential circuit to finite state machine

- Every counter and shift register in Unit 4 is a special case of a **finite state machine (FSM)**
- An FSM is described by five things: a finite set of **states**, a set of **inputs**, a set of **outputs**, a **next-state function** (present state + input → next state) and an **output function**
- What the output function may depend on is the one design choice that splits every FSM into one of two standard models

The Moore model

- In a **Moore machine** the output depends only on the **present state**, never directly on the input
- The flip-flops of the state register hold an n-bit code that identifies the current state
- The output logic reads only that state
- So the output is stable for the whole clock period and changes only right after a clock edge, in sync with the state

Moore machine

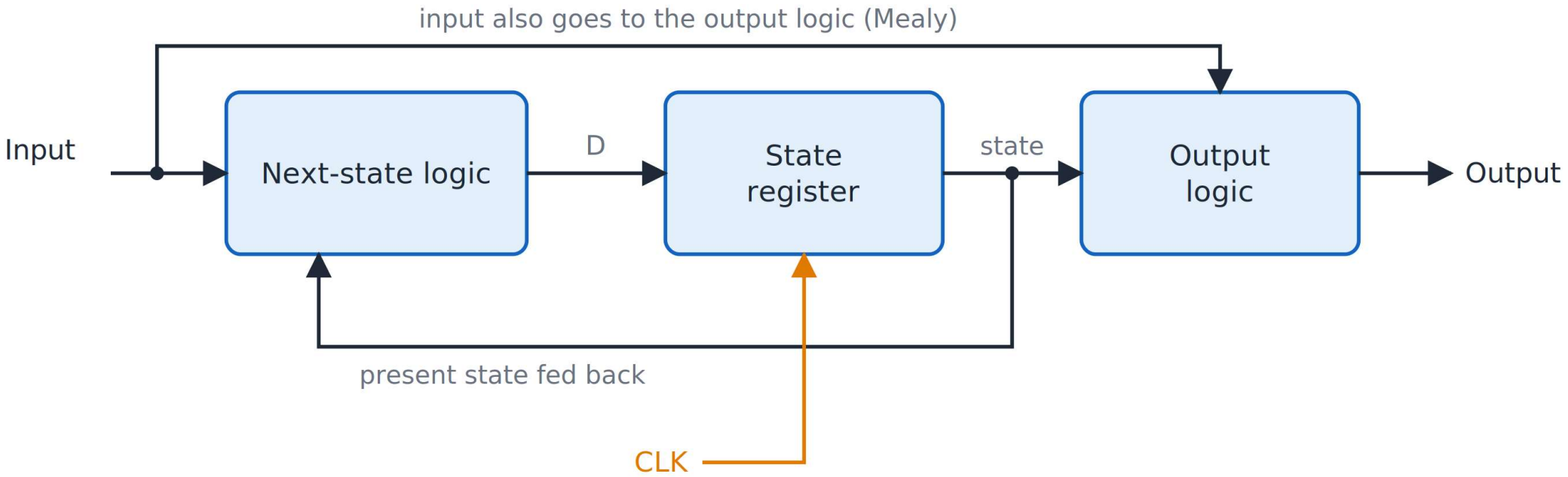


Moore machine: the output logic sees only the present state.

The Mealy model

- A **Mealy machine** adds one wire: the output logic also reads the **current input** directly, not just the state
- The output logic is purely combinational
- So a Mealy output can change the instant the input changes, even between clock edges
- That is why Mealy outputs are **not** guaranteed stable for a full clock cycle, unlike Moore outputs

Mealy machine

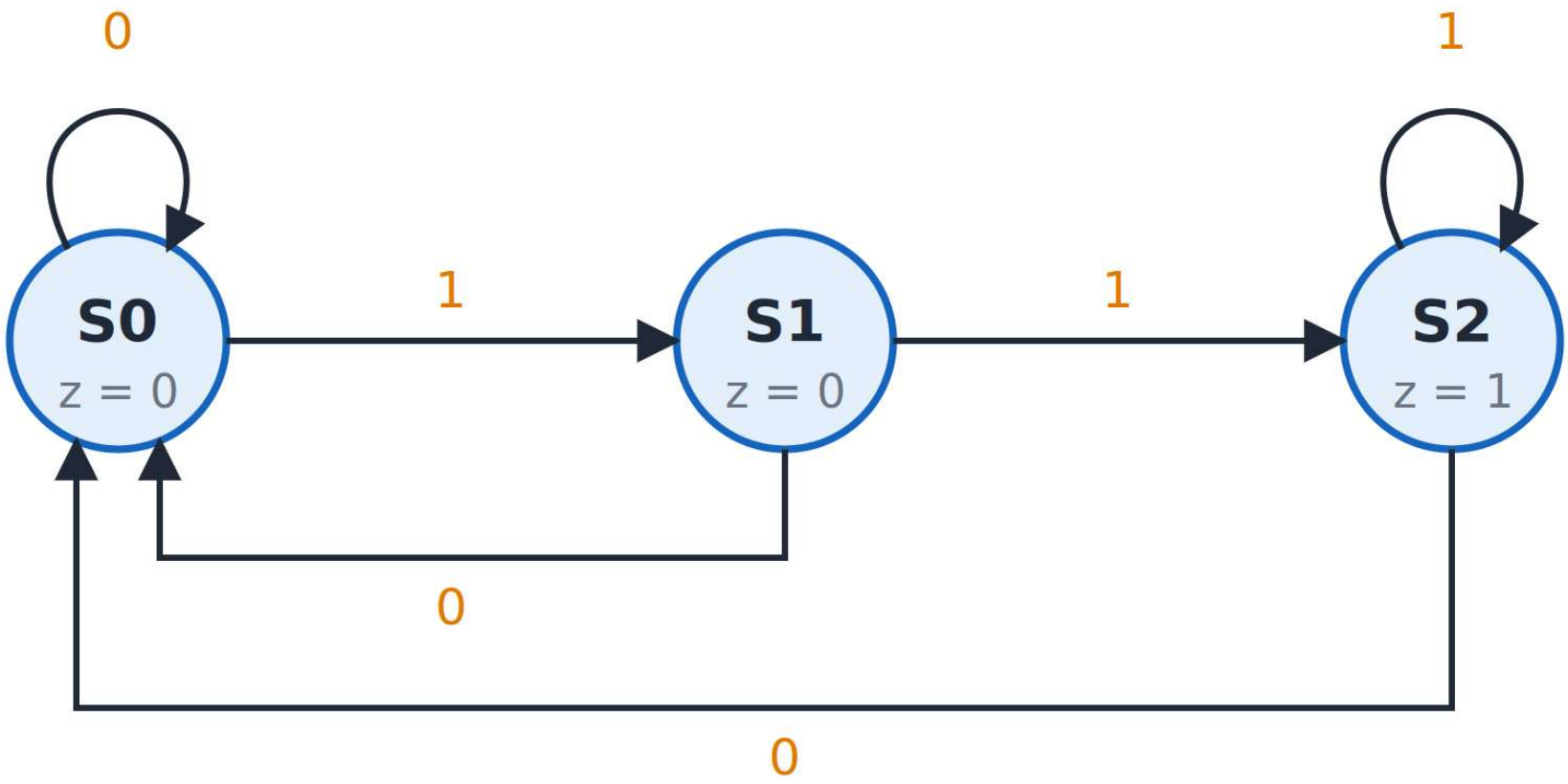


Mealy machine: the output logic also sees the input.

Moore version: detecting two consecutive 1s (3 states)

- Build a circuit whose output goes to 1 whenever the last two bits on a serial input were both 1
- The two models solve this with a different number of states
- **S0** = “no 1 seen yet”, **S1** = “last bit was 1”, **S2** = “last two bits were 11”
- S2 is the only state whose output z is 1
- The machine stays in S2 (self-loop on input 1) so the output keeps reading 1 across a longer run of 1s

Moore detector: three states

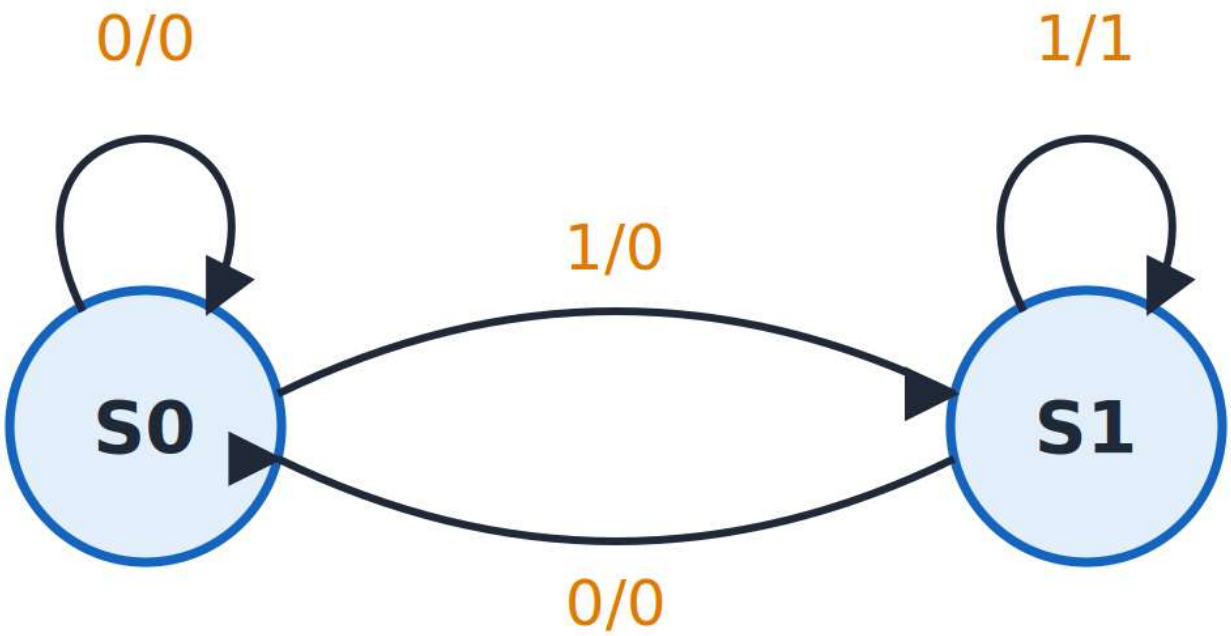


Moore detector for two consecutive 1s: three states, output z shown in each state.

Mealy version: detecting two consecutive 1s (2 states)

- Each arrow is labelled **input/output**
- **S0** = “last bit wasn’t 1” (or no history yet), **S1** = “last bit was 1”
- The output is 1 only on the transition S1 → S1 driven by input 1: the second 1 of a “11” pair
- It reverts to 0 as soon as the input changes
- No dedicated third state is needed to remember that the output should be high

Mealy detector: two states



Mealy detector for two consecutive 1s: two states, arrows labelled input/output.

Moore versus Mealy

Property	Moore	Mealy
Output depends on	State only	State and input
States needed (this example)	3	2
Output changes	Only at clock edges	Can change mid-cycle with the input
Output stability	Stable for the full clock period	Can glitch if input is noisy

5.2 State Diagrams & State Tables

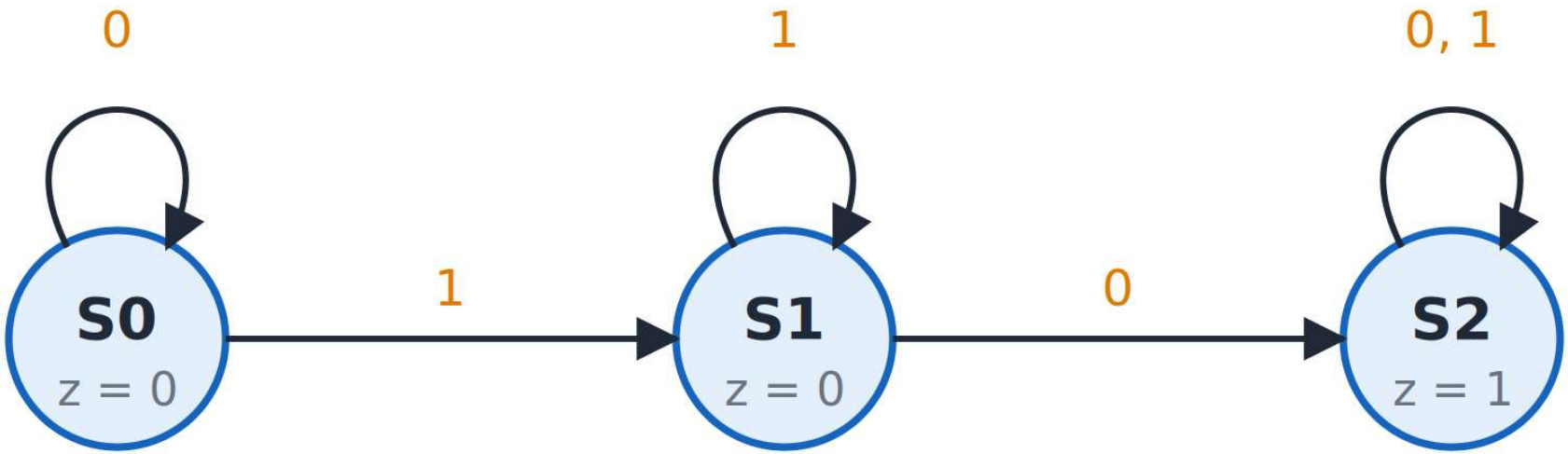
IN SIMPLE TERMS

A state diagram draws the machine as circles and arrows; a state table lists the same information row by row. One is easy to read, the other easy to design from.

State table format

- A state table has **one row per state**
- **Moore:** Present State | Next State for each input value | Output (one value per state)
- **Mealy:** Present State | Next State / Output pair for each input value
- A Moore table has the output as its own column, because the output depends only on the state
- A Mealy table folds the output into each next-state cell, because the output depends on the transition
- The diagram is easier to read; the table is what you build excitation tables and K-maps from

Moore example: a 1 followed later by a 0



Moore detector for a 1 followed later by a 0.

Diagram → table: a Moore example

S0 = “haven’t seen a 1 yet”, S1 = “saw a 1, waiting for a 0”, S2 = “saw 1-then-0: done”. S2’s self-loop is labelled “0,1” because either input keeps it in S2.

Present state	Next state (x=0)	Next state (x=1)	Output z
S0	S0	S1	0
S1	S2	S1	0
S2	S2	S2	1

Diagram → table: the rule

- For each state (row), follow the arrow leaving it for every possible input value
- Write down where that arrow lands: the next-state column
- For a Moore machine, copy the state's own output into the output column
- The conversion is entirely mechanical

Table → diagram: a Mealy example

The “detect two consecutive 1s” Mealy machine from the previous topic, given as a table first.

Present state	Next state/Output (x=0)	Next state/Output (x=1)
S0	S0 / 0	S1 / 0
S1	S0 / 0	S1 / 1

Table → diagram: redrawing the Mealy machine

- Draw one circle per row: S0 and S1
- For each cell draw an arrow from the row's state to the next state named in the cell, labelled **input/output**
- **Row S0:** x = 0 cell "S0/0" → self-loop on S0 labelled 0/0; x = 1 cell "S1/0" → arrow S0 → S1 labelled 1/0
- **Row S1:** x = 0 cell "S0/0" → arrow S1 → S0 labelled 0/0; x = 1 cell "S1/1" → self-loop on S1 labelled 1/1
- This is exactly the two-state diagram from the previous topic, rebuilt from the table with no guessing

5.3 State Reduction & State Assignment

IN SIMPLE TERMS

If two states always behave the same way, merge them; then give each remaining state a binary code for the flip-flops.

Why reduce states first?

- A state table built directly from a word problem often has **more states than the behaviour strictly requires**
- Sometimes two states are completely indistinguishable from the outside
- Every state costs flip-flops, and every extra flip-flop costs excitation logic
- Merging redundant states before the design shrinks the hardware with **zero change** in observable behaviour

Equivalent states

- Two states are **equivalent** if, starting from either one, every possible input sequence gives exactly the same output sequence
- States P and Q are equivalent if:
- **1.** Their outputs match for every input (immediately for Mealy; their own state output for Moore), **and**
- **2.** For every input value, the next states they go to are themselves equivalent (possibly recursively)
- In practice: group states by matching output, then compare next-state columns within each group; merge, and repeat until no more merges are possible

Worked example: a 5-state table

Reduce 5 states to 3.

State	Next (x=0)	Next (x=1)	Output
A	B	C	0
B	D	E	1
C	D	E	1
D	D	D	0
E	D	D	0

Worked example: merging states

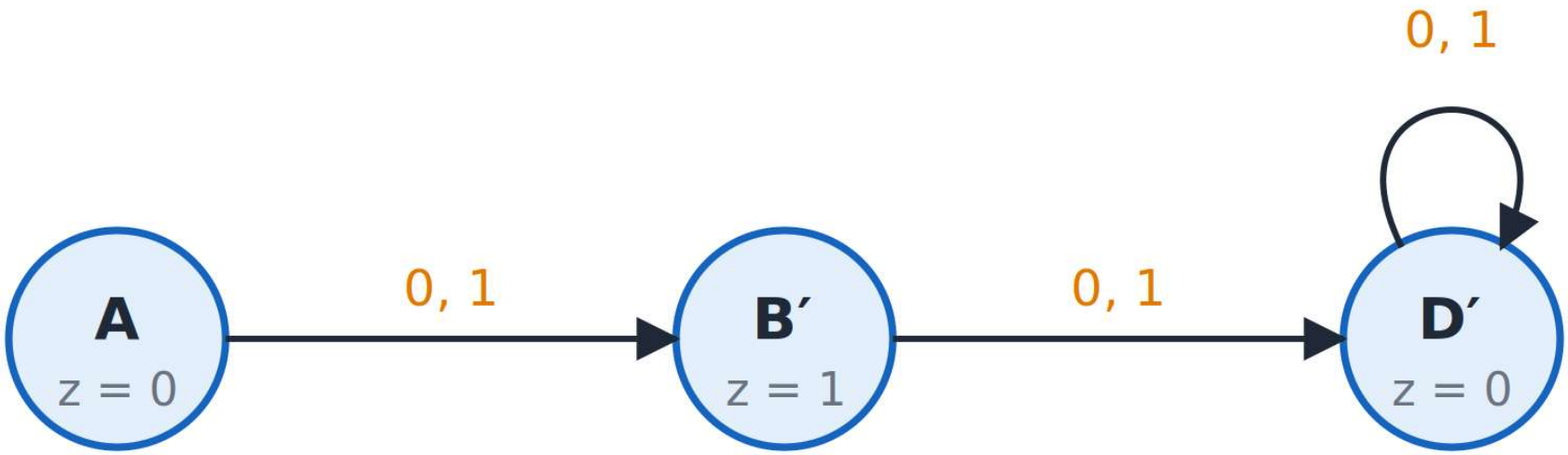
- Group by output first: **{A, D, E}** have output 0 and **{B, C}** have output 1
- Within {B, C}: both rows read exactly (D, E), so **B ≡ C**
- Within {A, D, E}: A’s row is (B, C), but D’s and E’s rows both read (D, D), so **D ≡ E**
- A stays on its own
- Merging B with C (call it B′) and D with E (call it D′) gives the reduced table

The reduced table

Five states collapsed to three. This table describes exactly the same input/output behaviour as the original.

State	Next (x=0)	Next (x=1)	Output
A	B′	B′	0
B′	D′	D′	1
D′	D′	D′	0

The reduced three-state machine



The reduced three-state machine.

State assignment

- A flip-flop stores a bit, not a letter: every remaining state needs a unique **binary code**
- For k states, the minimum number of flip-flops is $\lceil \log_2 k \rceil$: 3 states need 2 flip-flops (one code left unused)
- Which code goes to which state is a free choice that **never changes** the external behaviour
- The choice changes how simple the excitation equations turn out; the optimal assignment is a hard search in general
- Real designs use a straightforward sequential assignment (as in this course) or CAD tool heuristics

Worked example: assigning codes

Code 11 is unused: a don’t-care input combination for the excitation K-maps.

State	Code (Q_1Q_0)
A	00
B′	01
D′	10

The reduced table in binary


```
Q1Q0 = 00 (A):   Next (x=0) = 01,  Next (x=1) = 01,  Output = 0
Q1Q0 = 01 (B'):  Next (x=0) = 10,  Next (x=1) = 10,  Output = 1
Q1Q0 = 10 (D'):  Next (x=0) = 10,  Next (x=1) = 10,  Output = 0
```

- This is the form the next topic's excitation tables and K-maps start from

5.4 FSM Design Procedure: Sequence Detector

IN SIMPLE TERMS

To build any state machine, go diagram → table → state codes → flip-flop input table → K-maps → gates.

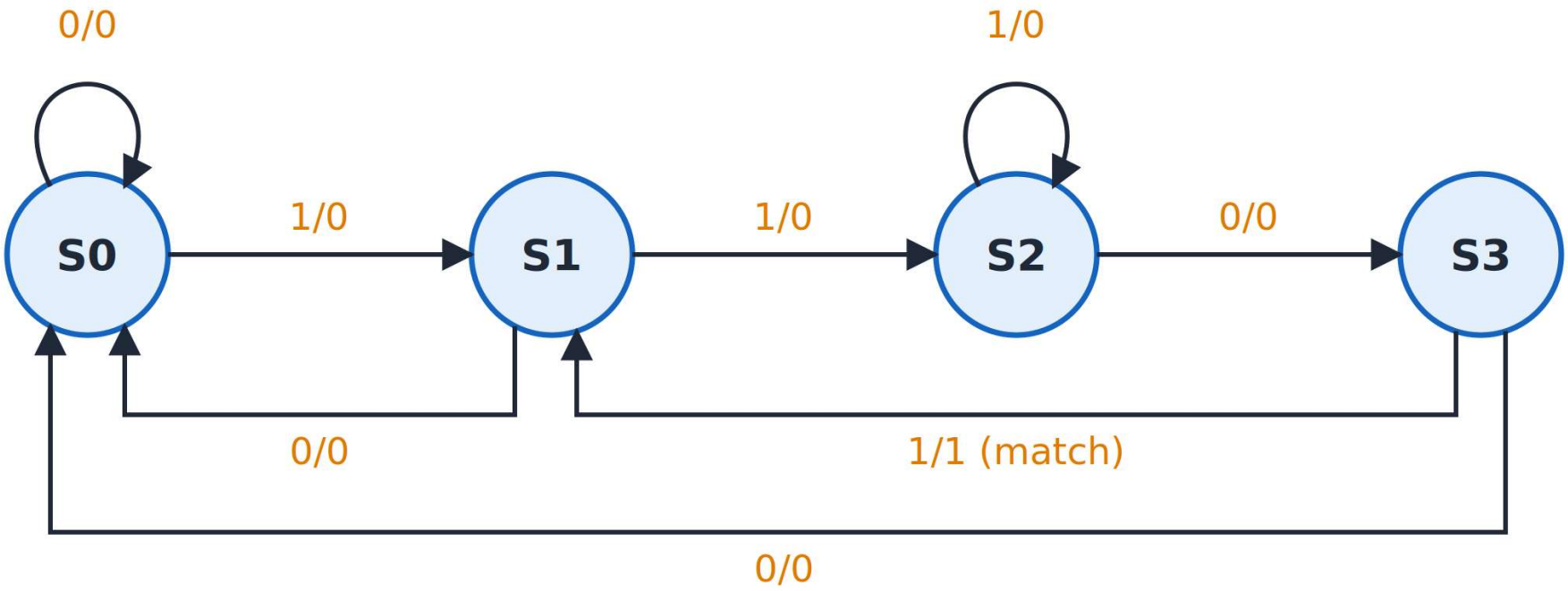
Problem: the overlapping 1101 detector

- Design a serial circuit that flags every occurrence of the bit pattern **1101** in an input stream
- Overlapping occurrences count: in 1101101 both the first four bits and the last four bits are valid matches, sharing one bit
- The procedure: state table → excitation table → K-map → circuit, as for counters

Step 1: state diagram

- Use a **Mealy** machine: each state tracks how much of “1101” has been matched by the tail of the input
- On a mismatch, do not necessarily fall back to “nothing matched”: check whether the tail still matches a shorter prefix of the pattern
- **S0** = matched “”, **S1** = matched “1”, **S2** = matched “11”, **S3** = matched “110”
- On input 1, **S3** outputs 1 (match) and jumps back to **S1**, not S0: the bit that completed “1101” is itself the start of a possible new match
- From S3 on a 0, and from S1 on a 0, the machine falls all the way back to S0

Mealy state diagram of the 1101 detector



Mealy state diagram of the overlapping 1101 detector.

Full state table

Present state	Next (x=0)	Next (x=1)	Output (x=0)	Output (x=1)
S0	S0	S1	0	0
S1	S0	S2	0	0
S2	S3	S2	0	0
S3	S0	S1	0	1

Step 2: state assignment

- Four states need $\lceil \log_2 4 \rceil = 2$ flip-flops
- Use straightforward sequential assignment (from the previous topic): **S0 = 00, S1 = 01, S2 = 10, S3 = 11**
- Call the two state bits **A** (MSB) and **B** (LSB)
- Use **JK** flip-flops

Step 3: excitation table

A	B	x	A^+	B^+	J_A	K_A	J_B	K_B	y
0	0	0	0	0	0	X	0	X	0
0	0	1	0	1	0	X	1	X	0
0	1	0	0	0	0	X	X	1	0
0	1	1	1	0	1	X	X	1	0
1	0	0	1	1	X	0	1	X	0
1	0	1	1	0	X	0	0	X	0
1	1	0	0	0	X	1	X	1	0
1	1	1	0	1	X	1	X	0	1

Step 4: K-map for J of flip-flop A

Each of J_A, K_A, J_B, K_B is its own 3-variable function of (A, B, x).

A \ Bx	00	01	11	10
0	0	0	1	0
1	X	X	X	X

Step 4: K-map for K of flip-flop A

A \ Bx	00	01	11	10
0	X	X	X	X
1	0	0	1	1

Step 4: K-map for J of flip-flop B

A \ Bx	00	01	11	10
0	0	1	X	X
1	1	0	X	X

Step 4: K-map for K of flip-flop B

A \ Bx	00	01	11	10
0	X	X	1	1
1	X	X	0	1

Step 4: the equations

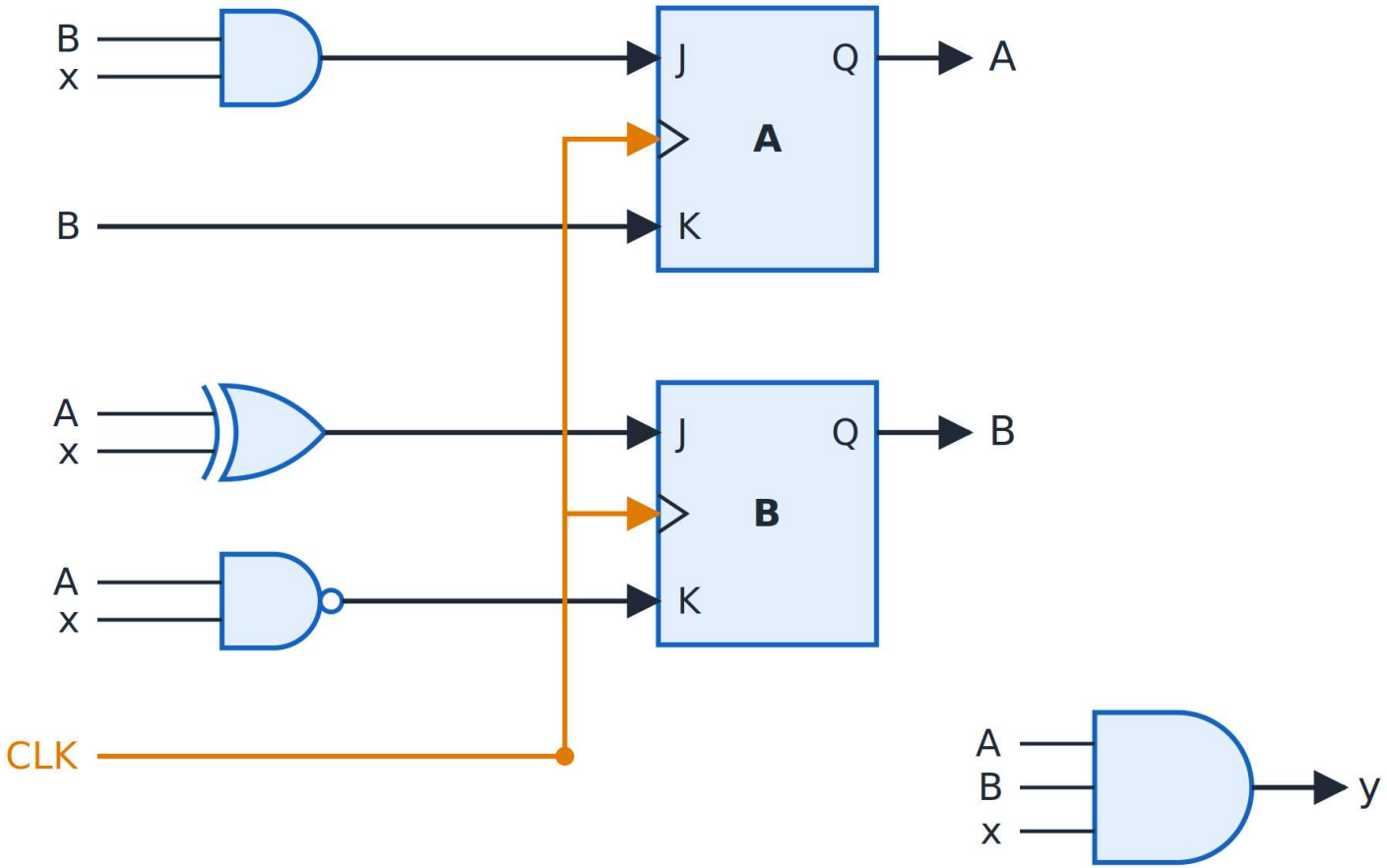
$$J_A = Bx$$
$$K_A = B$$
$$J_B = A'x + Ax'$$
$$K_B = A' + x'$$
$$y = ABx$$

- The output y needs no map: a single required 1 at A = 1, B = 1, x = 1, with no don’t-cares to exploit

Step 5: the circuit

- Two JK flip-flops (A, B) hold the state
- A small block of combinational logic, built from the five equations, drives the J/K inputs and the output
- The **J, K logic** block: $J_A = Bx$, $K_A = B$, $J_B = A'x + Ax'$, $K_B = A' + x'$
- The **output logic** is the single gate $y = ABx$, reading the state and x directly
- That is why this is a Mealy machine and not a Moore one

Circuit of the 1101 detector



Signals with the same name are connected

Circuit of the 1101 detector: $J_A = Bx$, $K_A = B$, $J_B = A \oplus x$, $K_B = (Ax)'$, $y = ABx$.

5.5 Programmable Logic Devices

IN SIMPLE TERMS

Programmable logic chips contain a grid of AND and OR gates; you program which connections to keep. ROMs, PLAs and PALs differ in which of the two grids is programmable.

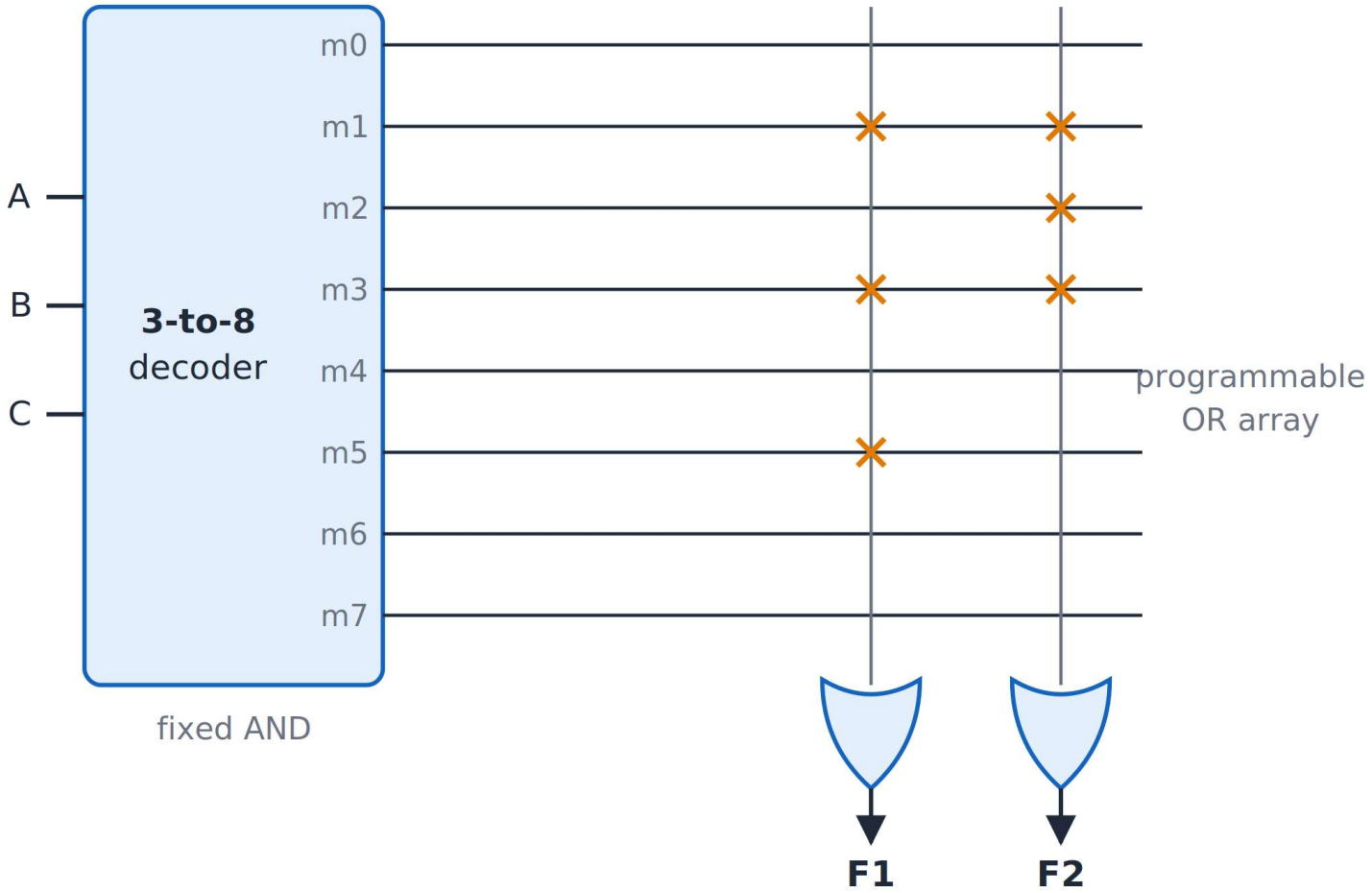
The AND-OR structure

- Any Boolean function is a canonical sum of minterms: a layer of AND gates producing product terms, feeding a layer of OR gates
- A **programmable logic device (PLD)** is that AND-OR structure built as reconfigurable hardware
- Instead of wiring discrete gates for one function, you “program” (or the manufacturer pre-wires) which connections exist
- The three classic families, **ROM**, **PLA** and **PAL**, differ in one thing: which of the two arrays, if either, is fixed or programmable

ROM: fixed AND, programmable OR

- A ROM used as combinational logic treats its n address inputs as a full decoder
- The **AND array is fixed** and always generates all 2^n minterms, whether or not a function needs them
- Only the **OR array is programmable**: each output sums any subset of the minterms
- This is a truth table in hardware: address in, the row's data out
- **Cost:** size grows as 2^n: practical for few inputs, exponentially wasteful for many

ROM

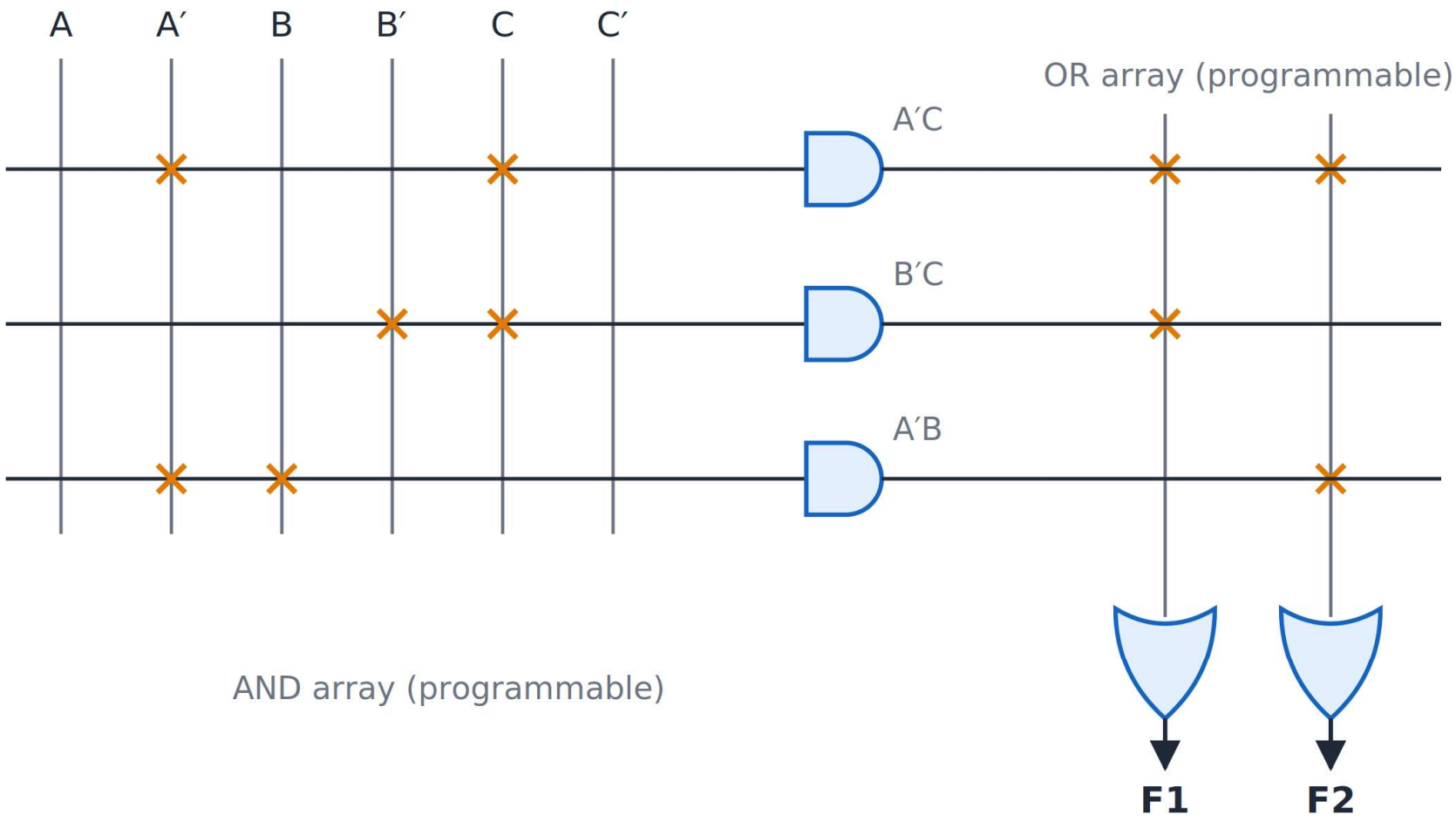


ROM: a fixed decoder makes every minterm; the programmable OR array picks m1, m3, m5 for F1 and m1, m2, m3 for F2.

PLA: programmable AND, programmable OR

- A **Programmable Logic Array** makes **both layers** programmable
- The AND array generates only the product terms some output actually uses, not all 2^n
- A product term computed once can feed **multiple OR gates**, so outputs that share a term do not pay for it twice
- It is the most flexible of the three
- The programmable OR array is more complex and historically more expensive to make reliably than a fixed one

PLA

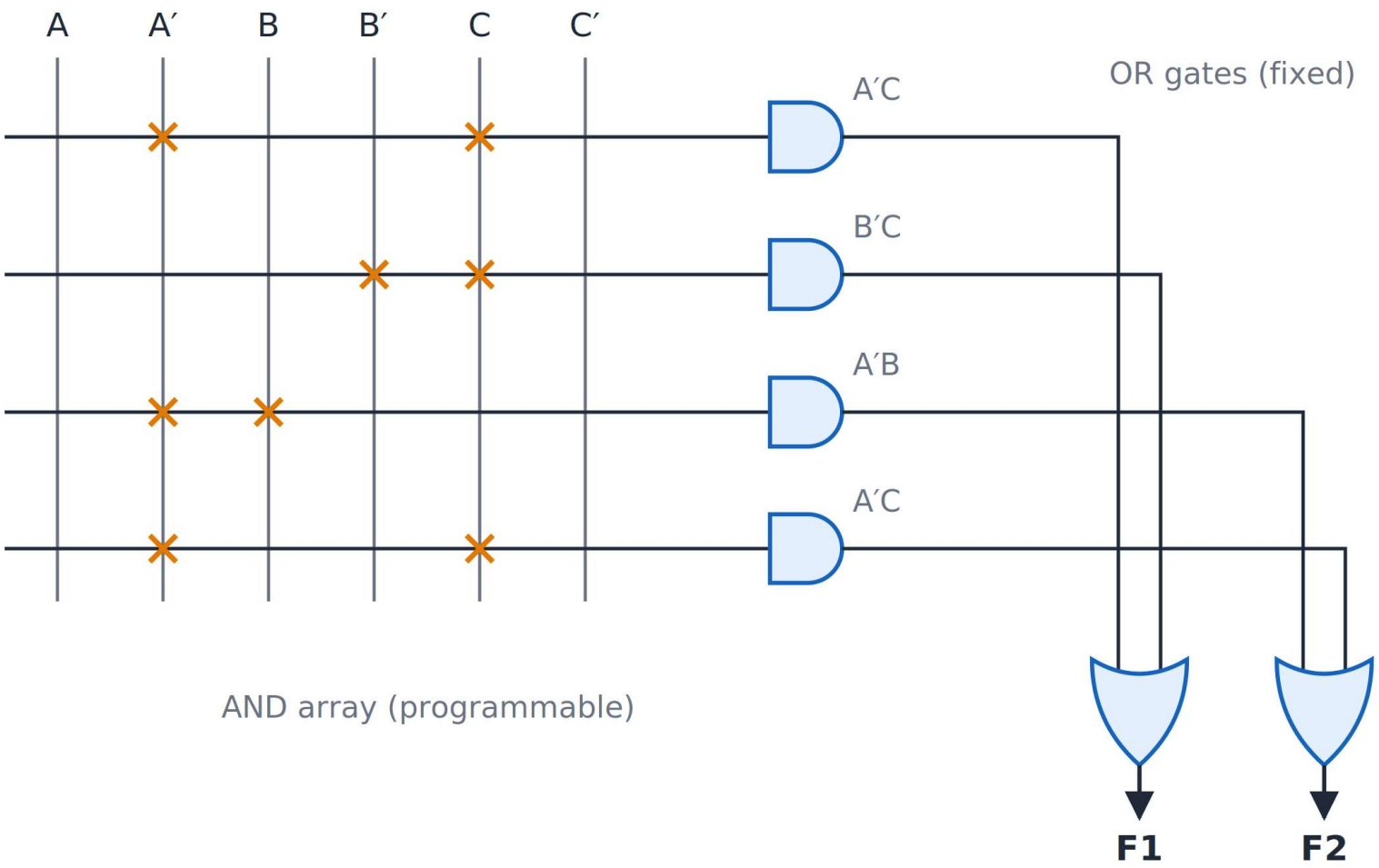


PLA: three product terms, with A’C shared by both outputs.

PAL: programmable AND, fixed OR

- A **Programmable Array Logic** device programs only the **AND array**
- The OR array is **fixed at manufacture**: each output has its own dedicated, fixed-size bank of AND-array lines
- No programmable OR crosspoints makes the PAL cheaper and faster than an equivalent PLA
- **Trade-off:** a product term needed by two outputs must be generated **twice**, once in each output's bank

PAL



PAL: fixed OR gates, so A’C must be built twice.

ROM, PLA and PAL compared

Device	AND array	OR array	Product-term sharing
ROM	Fixed (all 2^n)	Programmable	Free (every term always exists)
PLA	Programmable	Programmable	Free (any term can feed any output)
PAL	Programmable	Fixed	None (each output needs its own copy)

Worked example: efficiency comparison


```
F1(A,B,C) = Σm(1,3,5)        F2(A,B,C) = Σm(1,2,3)
Minterms 1 and 3 are needed by both functions
ROM: the fixed decoder builds all 8 minterms → 8 AND lines
PLA: three distinct terms A'C, B'C, A'B; A'C feeds both ORs → 3 AND
lines
PAL: F1 needs A'C and B'C; F2 needs A'B and A'C; A'C built twice → 4
AND lines
```

- PLA (3) beats PAL (4) beats ROM (8) here, because the example has shared minterms and does not need every possible one

5.6 Memory: RAM and ROM

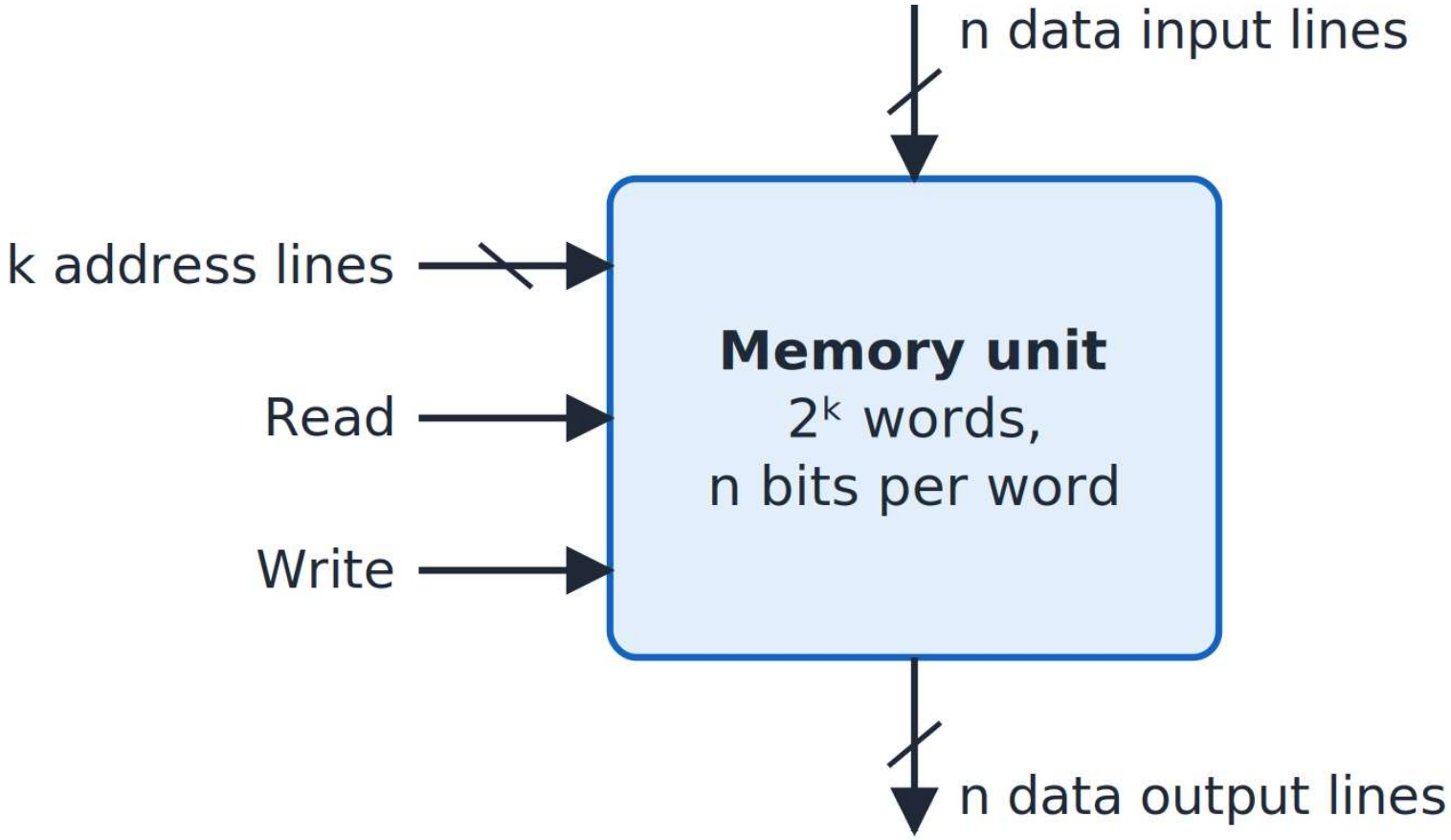
IN SIMPLE TERMS

A memory is a large set of numbered boxes, each holding one word. Give it an address and it either stores the word you supply (write) or hands back the word already there (read).

What a memory unit is

- A **memory unit** is a collection of binary storage cells plus the circuits that move information in and out
- It stores information in groups of bits called **words**; each word has an **address** from 0 to $2^k - 1$
- A memory of 2^k words of n bits has **k address lines**, **n data input lines**, **n data output lines** and control inputs for read or write
- Memories are named by size: 1K = 2^{10} = 1,024 words, 1M = 2^{20}, 1G = 2^{30}
- A "1K × 16" memory has 1,024 words of 16 bits: 10 address lines and 16 data lines in each direction

Block diagram of a memory unit



Block diagram of a memory unit. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 7.2.

Worked example: counting lines

Rule: 2^k = number of words → k = $\log_2$(words).

Memory	Words	Address lines (k)	Data lines (n)	Capacity
1K × 16	1,024	10	16	2,048 bytes
4K × 8	4,096	12	8	4,096 bytes
64K × 16	65,536	16	16	131,072 bytes

Write and read operations

- **Write:** (1) put the binary address on the address lines; (2) put the data bits on the data input lines; (3) activate the write control
- The word is stored at that address, replacing what was there
- **Read:** (1) put the address on the address lines; (2) activate the read control
- The stored word appears on the data output lines; reading does **not** change it
- Memory chips usually have a **memory enable** (chip select) input and one **Read/Write** line

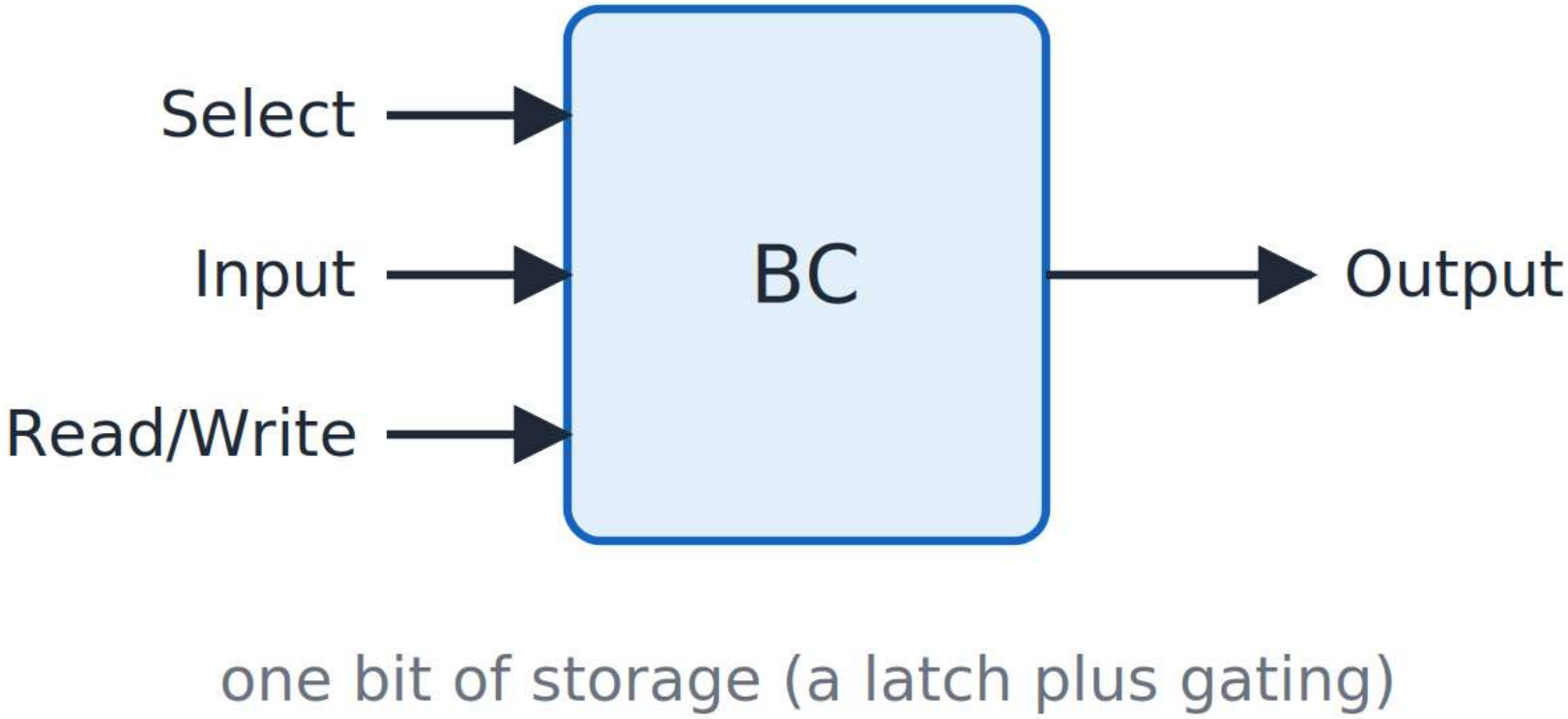
Memory enable and Read/Write

Memory enable	Read/Write	Operation
0	X	None (chip not selected)
1	0	Write to the selected word
1	1	Read from the selected word

Inside a RAM

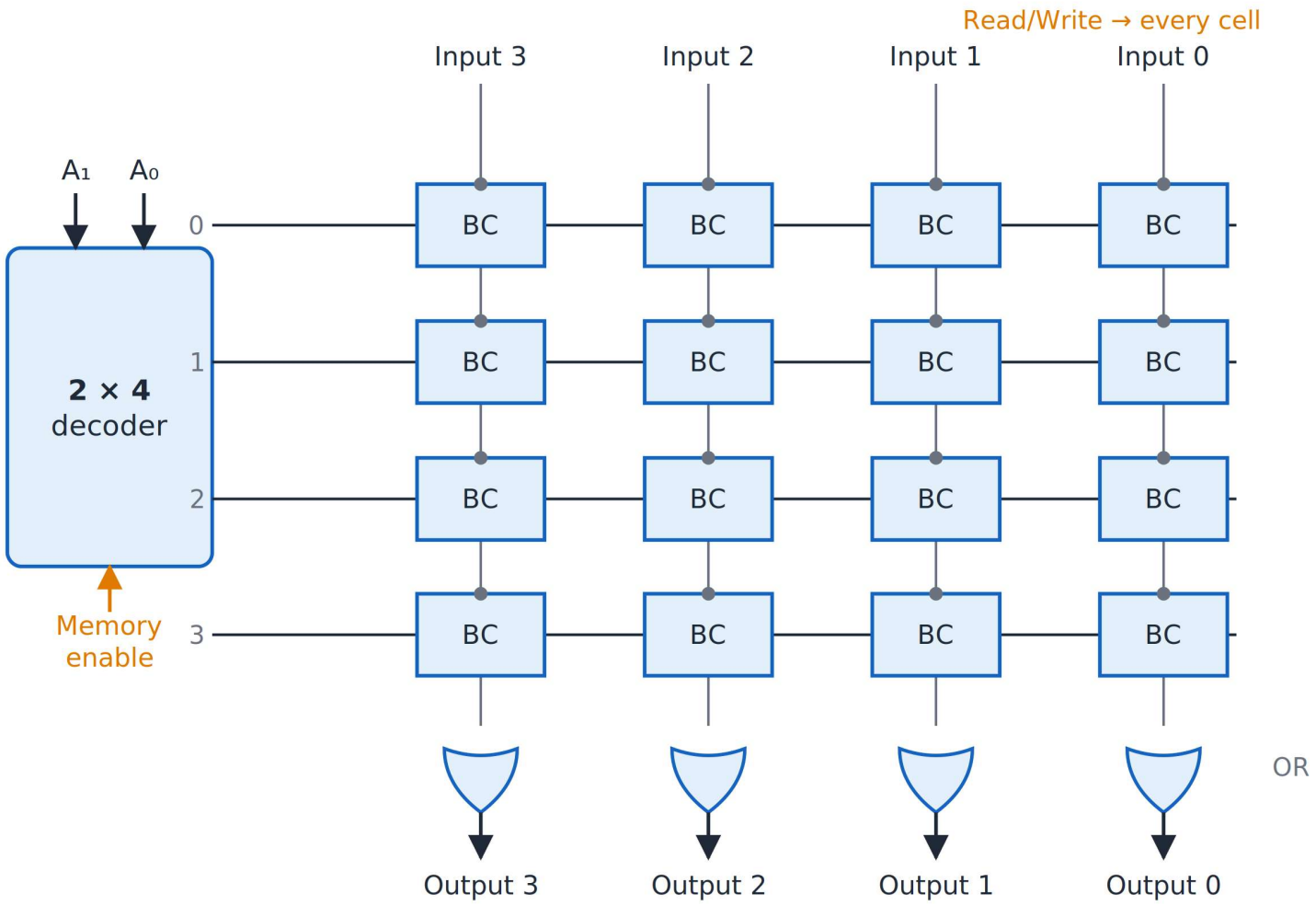
- Each bit is held in a **binary cell (BC)**: in effect a latch with gating
- When its select line is 1, Read/Write = 0 stores the input bit and Read/Write = 1 places the stored bit on the output
- A RAM of m words × n bits is an **m × n array of cells**
- A k-to-2^k **decoder** turns the address into one word-select line; every cell in that row is selected
- The data inputs go to every column, and an **OR gate per column** collects the selected row's bits for output

A binary storage cell



A binary storage cell as a block.

A 4 × 4 RAM

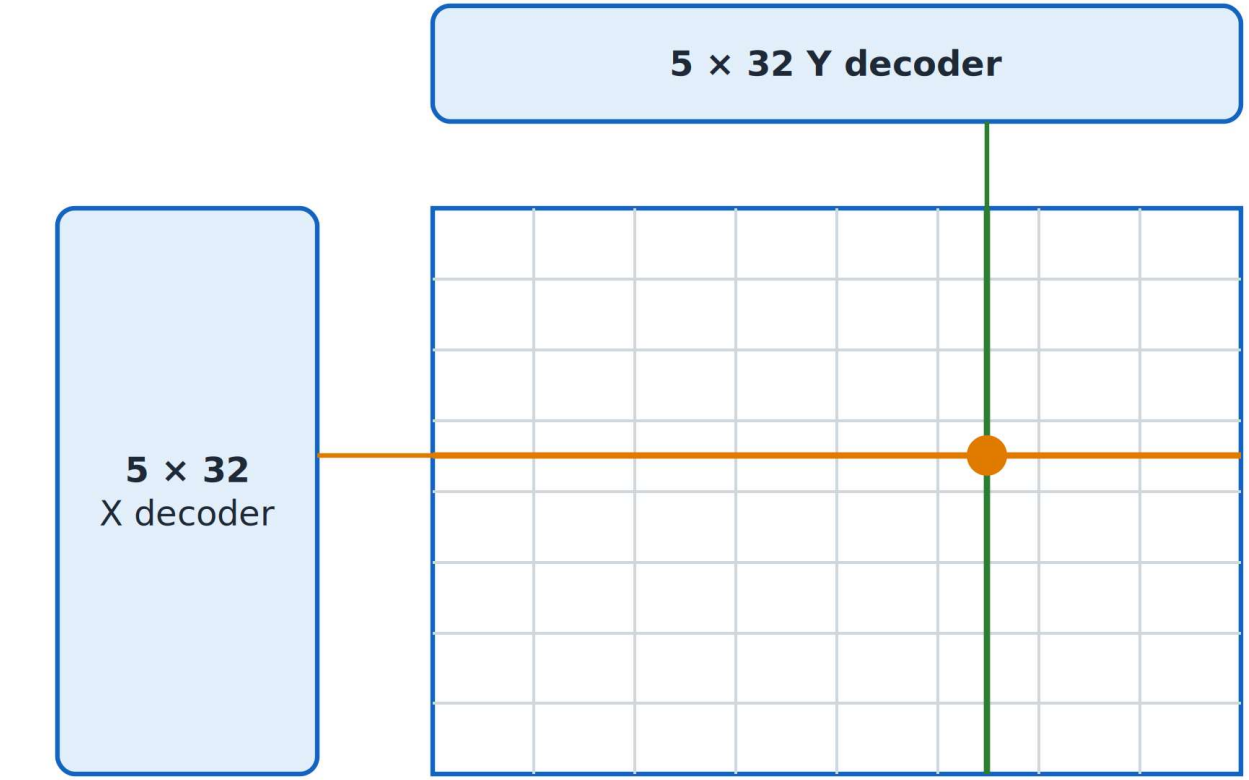


A 4 × 4 RAM: the 2 × 4 decoder selects one word of four cells. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 7.6.

Coincident (two-dimensional) decoding

- A single decoder for a 1K memory needs 1,024 outputs (1,024 AND gates)
- Arrange the words as a **32 × 32 grid** instead
- Two **5 × 32 decoders** do the job: one picks the row (X), the other the column (Y)
- The word where they cross is selected: only **64 decoder outputs**
- 1-D: one 10 × 1024 decoder → 1,024 outputs; 2-D: two 5 × 32 decoders → 64 outputs (32 + 32)

Two-dimensional decoding of a 1K memory



Two-dimensional decoding of a 1K memory: address 404 = 01100 10100 selects row 12 and column 20. Adapted from M. Morris Mano and Michael D. Ciletti, Digital Design, Fig. 7.7.

Types of memory

Access time: the time to select a word and read it. **Cycle time:** the time to complete one read or write before the next can start.

Type	Keeps data without power?	How it stores a bit	Typical use
SRAM (static RAM)	No (volatile)	Latch: holds while powered	Cache memory; faster
DRAM (dynamic RAM)	No (volatile)	Charge on a capacitor: must be refreshed every few milliseconds	Main memory; denser, cheaper per bit
ROM (mask-programmed)	Yes (non-volatile)	Fixed at manufacture	Fixed programs and tables
PROM	Yes	Programmed once by the user (fuses)	Small fixed tables
EPROM	Yes	Erased with ultraviolet light, reprogrammed	Development work
EEPROM / Flash	Yes	Erased and rewritten electrically	Firmware, USB drives, SSDs

5.7 FSM Case Studies: Traffic Light & Vending Machine

IN SIMPLE TERMS

A traffic light and a vending machine are both just a few states and a rule for moving between them; the same design steps as the sequence detector turn each into flip-flops and gates.

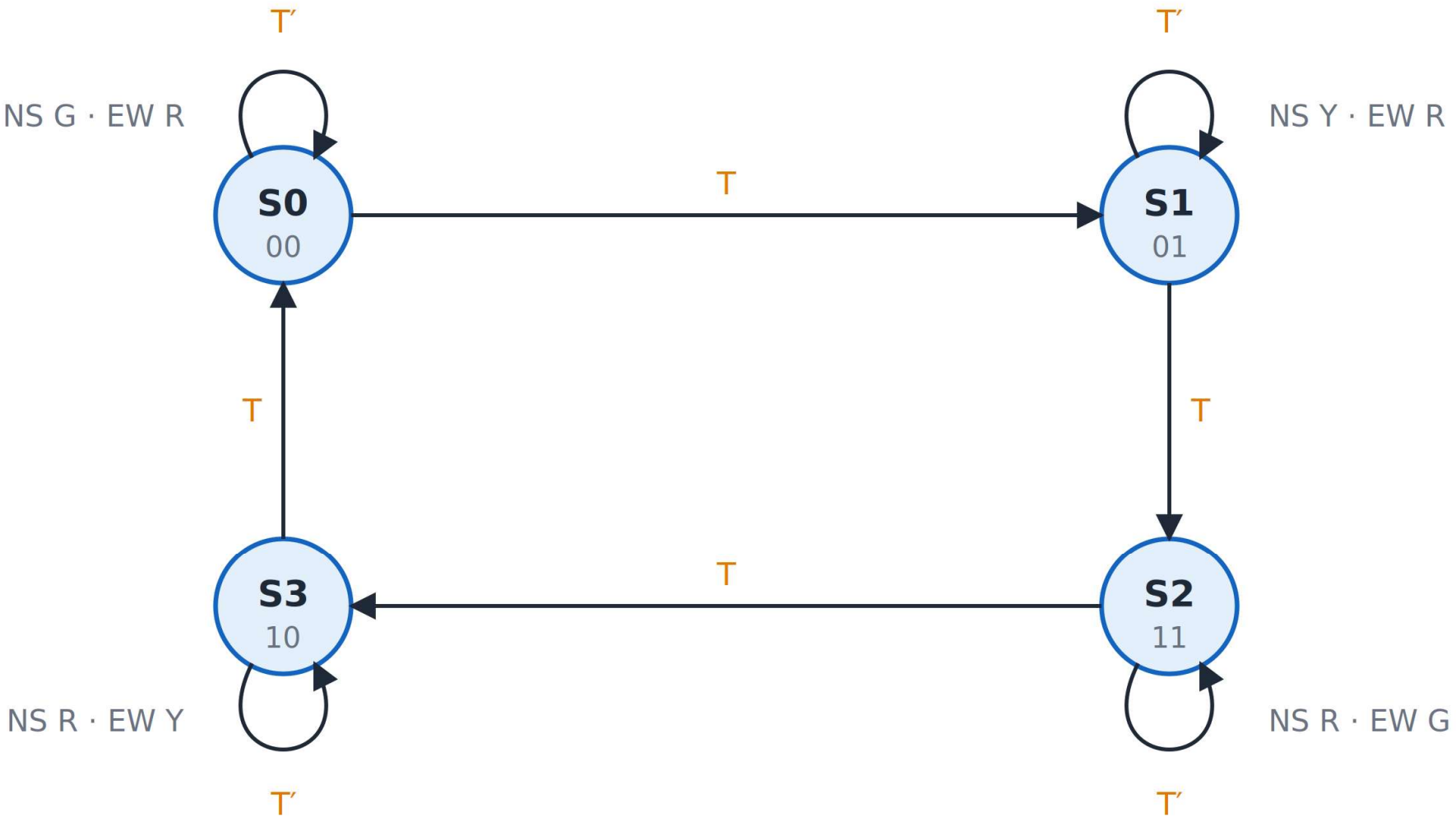
Two case studies, one procedure

- Procedure: state diagram → state table → state assignment → flip-flop input equations → output equations
- Both are **Moore** machines: the outputs belong to the states
- Both use **D flip-flops**, so each D input simply equals the next-state bit

Case study 1: traffic-light controller

- A junction of a north–south (NS) road and an east–west (EW) road
- A timer input **T** becomes 1 when the current light has been on long enough
- Sequence: NS green → NS yellow → EW green → EW yellow → back to NS green
- The other road is red throughout

Traffic-light controller



Traffic-light controller: four states, each held while T = 0 and left when T = 1.

Traffic light: state table

State assignment (Gray code, so only one flip-flop changes per step): S0 = 00, S1 = 01, S2 = 11, S3 = 10 (Q_1Q_0).

Present Q_1Q_0	Next, T = 0	Next, T = 1	NS light	EW light
00 (S0)	00	01	Green	Red
01 (S1)	01	11	Yellow	Red
11 (S2)	11	10	Red	Green
10 (S3)	10	00	Red	Yellow

Traffic light: equations

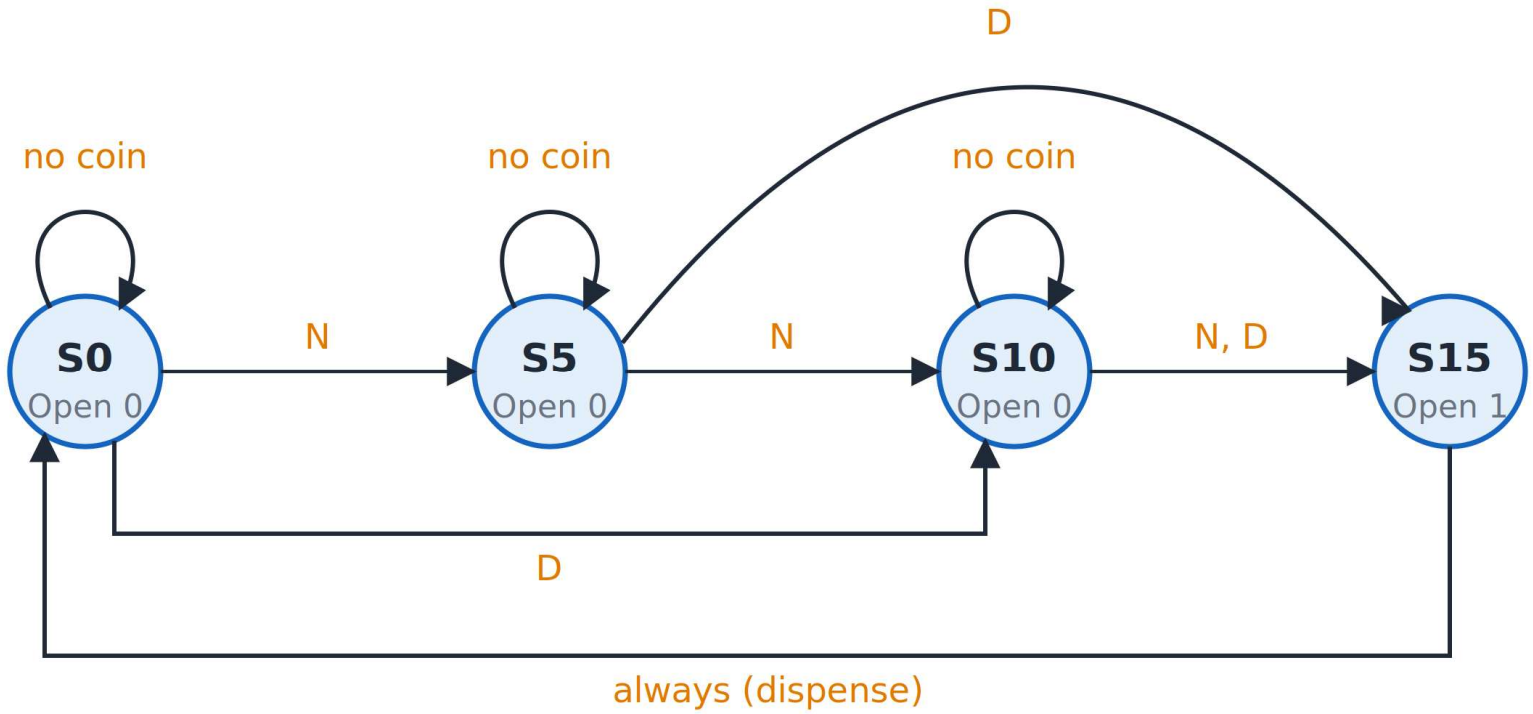

```
D1 = T'Q1 + TQ0
D0 = T'Q0 + TQ1'
NS green = Q1'Q0'      NS yellow = Q1'Q0      NS red = Q1
EW green = Q1Q0        EW yellow = Q1Q0'      EW red = Q1'
```

- With T = 0 each bit holds; with T = 1 Q_1 takes the old Q_0 and Q_0 takes the old Q_1'
- Check: in S2 (11), NS red = Q_1 = 1 and EW green = Q_1Q_0 = 1; every other lamp is 0 ✓

Case study 2: vending machine

- An item costs **₹15**
- The machine accepts ₹5 coins (input **N**) and ₹10 coins (input **D**), one coin per clock at most (N and D are never both 1)
- When ₹15 or more has been paid it opens (**Open** = 1) for one clock and returns to the start
- This simple version gives no change; adding a change output is a common extension

Vending machine



Vending machine: the state is the amount paid so far; S15 dispenses and returns to S0.

Vending machine: state table

State assignment: S0 = 00, S5 = 01, S10 = 10, S15 = 11 (Q_1Q_0).

Present	No coin	N (₹5)	D (₹10)	Open
S0 (00)	S0	S5	S10	0
S5 (01)	S5	S10	S15	0
S10 (10)	S10	S15	S15	0
S15 (11)	S0	S0	S0	1

Vending machine: equations


```
D1 = Q1'D + Q1Q0' + Q1'Q0N
D0 = Q0'N + Q1Q0'D + Q1'Q0N'
Open = Q1Q0
```

- From K-maps of Q_1, Q_0, N and D, with ND = 11 as don't-cares
- Check S5 + ₹10 (01, D = 1): $D_1 = Q_1'D = 1$, $D_0 = Q_1'Q_0N' = 1 \rightarrow 11 = S15$ ✓
- Check S15 (11): every term of D_1 and D_0 has Q_1' or Q_0', so both are 0 → S0 ✓

Unit 5 summary

- **5.1 Moore vs Mealy:** Moore output depends on the state only; Mealy output also on the input
- **5.2 State diagrams and tables:** the same machine in two forms; Moore has an output column, Mealy folds it into each cell
- **5.3 State reduction and assignment:** merge equivalent states; $\lceil \log_2 k \rceil$ flip-flops for k states
- **5.4 Sequence detector design:** diagram → table → codes → excitation table → K-maps → gates
- **5.5 PLDs:** ROM (fixed AND), PLA (both programmable), PAL (fixed OR)
- **5.6 Memory:** words and addresses, read and write, RAM structure, 2-D decoding, memory types
- **5.7 Case studies:** a traffic light and a vending machine, both Moore machines on D flip-flops

Review questions (1 of 2)

- **Q1.** State the difference between a Moore and a Mealy machine, and give an example where Mealy needs fewer states
- **Q2.** How does a Mealy state table differ from a Moore state table?
- **Q3.** When are two states equivalent? Reduce the 5-state table in the lesson to 3 states
- **Q4.** How many flip-flops are needed for k states, and why does the assignment not change the behaviour?

Review questions (2 of 2)

- **Q5.** List the steps of the FSM design procedure using the 1101 detector
- **Q6.** Compare ROM, PLA and PAL for F1 = Σm(1,3,5) and F2 = Σm(1,2,3)
- **Q7.** How many address and data lines does a 4K × 8 memory have?
- **Q8.** How does two-dimensional decoding reduce the decoder size of a 1K memory?

References

- **Textbook:** M. Morris Mano and Michael D. Ciletti, Digital Design
- **Adapted figures:**
- Block diagram of a memory unit (Fig. 7.2)
- A 4 × 4 RAM (Fig. 7.6)
- Two-dimensional decoding of a 1K memory (Fig. 7.7)